In [1]:
```
pip install mlxtend
```

```
Requirement already satisfied: mlxtend in d:\anaconda\lib\site-packages (0.23.0)
Requirement already satisfied: pandas>=0.24.2 in d:\anaconda\lib\site-packages (from mlxtend) (1.3.4)
Requirement already satisfied: joblib>=0.13.2 in d:\anaconda\lib\site-packages (from mlxtend) (1.3.2)
Requirement already satisfied: scipy>=1.2.1 in d:\anaconda\lib\site-packages (from mlxtend) (1.7.1)
Requirement already satisfied: matplotlib>=3.0.0 in d:\anaconda\lib\site-packages (from mlxtend) (3.4.3)
Requirement already satisfied: numpy>=1.16.2 in d:\anaconda\lib\site-packages (from mlxtend) (1.20.3)
Requirement already satisfied: scikit-learn>=1.0.2 in d:\anaconda\lib\site-packages (from mlxtend) (1.3.2)
Requirement already satisfied: cycler>=0.10 in d:\anaconda\lib\site-packages (from matplotlib>=3.0.0->mlxtend)
(0.10.0)
Requirement already satisfied: pyparsing>=2.2.1 in d:\anaconda\lib\site-packages (from matplotlib>=3.0.0->mlxte
nd) (3.0.4)
Requirement already satisfied: pillow>=6.2.0 in d:\anaconda\lib\site-packages (from matplotlib>=3.0.0->mlxtend)
(8.4.0)
Requirement already satisfied: kiwisolver>=1.0.1 in d:\anaconda\lib\site-packages (from matplotlib>=3.0.0->mlxt
end) (1.3.1)
Requirement already satisfied: python-dateutil>=2.7 in d:\anaconda\lib\site-packages (from matplotlib>=3.0.0->m
lxtend) (2.8.2)
Requirement already satisfied: six in d:\anaconda\lib\site-packages (from cycler>=0.10->matplotlib>=3.0.0->mlxt
end) (1.16.0)
Requirement already satisfied: pytz>=2017.3 in d:\anaconda\lib\site-packages (from pandas>=0.24.2->mlxtend) (20
21.3)
Requirement already satisfied: threadpoolctl>=2.0.0 in d:\anaconda\lib\site-packages (from scikit-learn>=1.0.2-
>mlxtend) (2.2.0)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
!pip install memory_profiler
```

```
Requirement already satisfied: memory_profiler in d:\anaconda\lib\site-packages (0.61.0)
Requirement already satisfied: psutil in d:\anaconda\lib\site-packages (from memory_profiler) (5.8.0)
```

In [3]:
```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
import time
from memory_profiler import memory_usage

# 加载CSV文件
file_path = "D:\\DeskTop\\Basket.csv"
df = pd.read_csv(file_path)

# 只选择商品列（包含yes和no的列）
basket_columns = ['Fruits & vegetables', 'Meat', 'Milk products', 'Canned vegetables', 'Canned meat',
                  'Frozen goods', 'Beer', 'Wine', 'Soda drinks', 'Fish', 'Textile']
basket_df = df[basket_columns]

# 将'yes'和'no'转换为True和False
basket_sets = basket_df.applymap(lambda x: True if x == 'Yes' else False)

# 使用Apriori算法
# 三个不同的最小支持度： 0.01, 0.05, 0.1
support_thresholds = [0.01, 0.05, 0.1]
for support in support_thresholds:
    start_time = time.time()
    mem_usage_before = memory_usage()[0]

    frequent_itemsets = apriori(basket_sets, min_support=support, use_colnames=True)

    mem_usage_after = memory_usage()[0]
    end_time = time.time()

    print(f"Support: {support}, Number of Frequent Itemsets: {len(frequent_itemsets)}")
    print( 'Execution Time:', end_time - start_time)
    print('Memory Usage:', mem_usage_after - mem_usage_before)

# 选择一个支持度进行关联规则分析
selected_support = 0.05
frequent_itemsets = apriori(basket_sets, min_support=selected_support, use_colnames=True)

# 生成关联规则
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.1)

# 按置信度降序排列
rules = rules.sort_values(by= 'confidence', ascending=False)

# 输出关联规则
print(rules)
```

```
Support: 0.01, Number of Frequent Itemsets: 273
Execution Time: 0.227783203125
Memory Usage: 1.29296875
Support: 0.05, Number of Frequent Itemsets: 57
Execution Time: 0.21723103523254395
Memory Usage: 0.0078125
Support: 0.1, Number of Frequent Itemsets: 17
Execution Time: 0.2183687686920166
Memory Usage: 0.01171875
                         antecedents                consequents  \
92           (Canned vegetables, Beer)             (Frozen goods)
91                (Frozen goods, Beer)        (Canned vegetables)
90   (Frozen goods, Canned vegetables)                     (Beer)
104                      (Fish, Beer)             (Frozen goods)
86                       (Fish, Beer)      (Fruits & vegetables)
..                                ...                        ...
101                    (Frozen goods)  (Canned vegetables, Fish)
27                     (Frozen goods)            (Milk products)
99                 (Canned vegetables)       (Frozen goods, Fish)
87               (Fruits & vegetables)                (Fish, Beer)
105                    (Frozen goods)                (Fish, Beer)

     antecedent  support  consequent  support  support  confidence      lift  \
92                 0.167                0.302    0.146    0.874251  2.894873
91                 0.170                0.303    0.146    0.858824  2.834401
90                 0.173                0.293    0.146    0.843931  2.880309
104                0.085                0.302    0.050    0.588235  1.947799
86                 0.085                0.301    0.050    0.588235  1.954270
..                   ...                  ...      ...         ...       ...
101                0.302                0.090    0.051    0.168874  1.876380
27                 0.302                0.177    0.051    0.168874  0.954091
99                 0.303                0.090    0.051    0.168317  1.870187
87                 0.301                0.085    0.050    0.166113  1.954270
105                0.302                0.085    0.050    0.165563  1.947799

     leverage  conviction  zhangs_metric
92   0.095566    5.550762       0.785788
91   0.094490    4.937083       0.779749
90   0.095311    4.530037       0.789377
104  0.024330    1.695143       0.531803
86   0.024415    1.697571       0.533661
..        ...         ...            ...
101  0.023820    1.094900       0.669139
27  -0.002454    0.990223      -0.064491
99   0.023730    1.094167       0.667567
87   0.024415    1.097271       0.698569
105  0.024330    1.096548       0.697135

[108 rows x 10 columns]
```

In [4]:

```
pip install pyfpgrowth
```

```
Requirement already satisfied: pyfpgrowth in d:\anaconda\lib\site-packages (1.0)Note: you may need to restart t
he kernel to use updated packages.
```

In [5]:

```
import pandas as pd
import pyfpgrowth
import time
from memory_profiler import memory_usage

# 加载CSV文件
file_path = "D:\\DeskTop\\Basket.csv"
df = pd.read_csv(file_path)

# 只选择商品列（包含yes和no的列）
basket_columns = ['Fruits & vegetables', 'Meat', 'Milk products', 'Canned vegetables', 'Canned meat',
                  'Frozen goods', 'Beer', 'Wine', 'Soda drinks', 'Fish', 'Textile']
basket_df = df[basket_columns]

# 将'yes'和'no'转换为True和False
basket_sets = basket_df.applymap(lambda x: True if x == 'Yes' else False)

# 将DataFrame转换为事务列表
transactions = basket_sets.apply(lambda x: list(x[x].index), axis=1).tolist()

# 使用FP-Growth算法
# 三个不同的最小支持度： 0.01, 0.05, 0.1
support_thresholds = [0.01, 0.05, 0.1]
for support in support_thresholds:
    start_time = time.time()
    mem_usage_before = memory_usage()[0]

    patterns = pyfpgrowth.find_frequent_patterns(transactions, support)

    mem_usage_after = memory_usage()[0]
    end_time = time.time()

    print(f"Support: {support}, Number of Frequent Patterns: {len(patterns)}")
    print( 'Execution Time:', end_time - start_time)
    print('Memory Usage:', mem_usage_after - mem_usage_before)

# 选择一个支持度进行关联规则分析
selected_support = 0.05
patterns = pyfpgrowth.find_frequent_patterns(transactions, selected_support)

# 生成关联规则
rules = pyfpgrowth.generate_association_rules(patterns, 0.1)  # 示例置信度阈值

# 将关联规则转换为DataFrame以便输出（可选）
rules_df = pd.DataFrame([(key, value[0], value[1]) for key, value in rules.items()],
                        columns=['Antecedents', 'Consequents', 'Confidence'])

# 按置信度降序排列
rules_df = rules_df.sort_values(by='Confidence', ascending=False)

# 输出关联规则
print(rules_df)
```

```
Support: 0.01, Number of Frequent Patterns: 943
Execution Time: 0.29895591735839844
Memory Usage: 0.32421875
Support: 0.05, Number of Frequent Patterns: 943
Execution Time: 0.25000643730163574
Memory Usage: 0.27734375
Support: 0.1, Number of Frequent Patterns: 943
Execution Time: 0.24903202056884766
Memory Usage: 0.140625
                                      Antecedents  \
0    (Canned meat, Canned vegetables, Milk products...
559  (Beer, Frozen goods, Fruits & vegetables, Milk...
609  (Canned meat, Canned vegetables, Meat, Soda dr...
607  (Canned meat, Canned vegetables, Fish, Meat, T...
606  (Canned meat, Canned vegetables, Fish, Meat, S...
..                                                 ...
731   (Beer, Canned vegetables, Frozen goods, Textile)
110  (Beer, Canned meat, Canned vegetables, Frozen ...
97   (Beer, Canned meat, Canned vegetables, Soda dr...
104     (Beer, Canned meat, Frozen goods, Soda drinks)
134                          (Beer, Soda drinks, Wine)

                                     Consequents  Confidence
0                                        (Wine,)    1.000000
559                                  (Fish, Wine)    1.000000
609                                      (Fish,)    1.000000
607                                 (Soda drinks,)    1.000000
```

```
606                                          (Textile,)    1.000000
..                                                  ...         ...
731                                           (Wine,)    0.115385
110                 (Fish, Fruits & vegetables, Wine)    0.111111
97    (Fish, Frozen goods, Fruits & vegetables, Wine)    0.111111
104  (Canned vegetables, Fish, Fruits & vegetables,...    0.111111
134           (Canned vegetables, Fish, Frozen goods)    0.100000

[906 rows x 3 columns]
```

分析和对比 Apriori 和 FP-Growth 算法

1. 算法结果比较（频繁项集数量）：

支持度	Apriori 频繁项集数量	FP-Growth 频繁项集数量
0.01	273	943
0.05	57	943
0.1	17	943

在所有给定的支持度水平下，FP-Growth 算法都找到了更多的频繁项集。这可能是因为 FP-Growth 算法在处理数据集时更为高效，也有可能是因为算法实现的差异导致了不同的项集被视为频繁项集。

2. 执行时间和内存使用：

支持度	Apriori 执行时间	FP-Growth 执行时间	Apriori 内存使用	FP-Growth 内存使用
0.01	0.2278秒	0.2990秒	1.2930MB	0.3242MB
0.05	0.2172秒	0.2500秒	0.0078MB	0.2773MB
0.1	0.0117秒	0.2490秒	0.0117MB	0.1406MB

FP-Growth 在内存使用上表现得更为高效，尤其是在低支持度下。然而，在这个特定的数据集和支持度设置中，Apriori 在高支持度阈值下的执行时间要优于 FP-Growth。这可能是因为当频繁项集数量减少时，Apriori 简单的候选生成和剪枝过程可能比 FP-Growth 的树构建和遍历过程更快。

对于两种算法的进一步思考

1.为什么在相同的最小支持度下，Apriori 和 FP-Growth 算法可能会发现不同数量的频繁项集?

Apriori 和 FP-Growth 算法在寻找频繁项集时采用了不同的策略。Apriori 是一种逐层搜索的方法，它从单个项开始，然后逐步增加项集的大小。在每一步，它都会剪枝掉那些不可能生成频繁项集的候选项集。这种方法可能会在早期阶段就剪枝掉一些实际上是频繁的项集。而 FP-Growth 是一种基于前缀树的方法，它在构建树的过程中保存了项集的全部信息，因此可以找到更多的频繁项集。但是，这也可能是由于实验设定或数据特性等因素导致的。

2.在实际电子商务场景中，如何选择合适的最小支持度和最小置信度?

最小支持度和最小置信度的选择主要取决于具体的业务需求和数据集的特性。一般来说，如果设置的最小支持度过低，可能会产生大量的频繁项集，这不仅会消耗大量的计算资源，而且得到的规则可能过于复杂，不易于理解和应用。反之，如果设置的最小支持度过高，可能会错过一些有价值的规则。最小置信度则影响规则的可靠性，过低的置信度可能会产生一些不准确的规则。因此，需要根据实际情况进行试验和调整，找到最适合的设置。

3.对于一个大型的电子商务平台，使用关联规则学习可能会遇到哪些挑战和限制?

- 数据量大：大型电子商务平台的交易数据量通常非常大，这会导致频繁项集挖掘的计算复杂度非常高，消耗大量的计算资源。
- 数据稀疏性：电子商务数据通常非常稀疏，即大多数商品只在少数交易中出现，这使得找到有意义的关联规则更加困难。
- 动态性：电子商务平台的交易数据是动态变化的，用户的购买行为和商品的销售情况都可能随时间变化，这要求关联规则学习算法能够适应这种动态性。
- 解释性和应用性：挖掘出的关联规则需要有良好的解释性，才能被业务人员理解和接受。此外，关联规则的应用也需要考虑实际的业务场景和约束，例如库存、物流等因素。